

- The embedded entities do not grow indefinitely.
- Relationships range from one to a few, between embedding and embedded entities.

Important considerations for MongoDB data Modelling

While designing database document structure and data models in MongoDB, special consideration should be given to the following aspects for deploying highly scalable, performance-centric and efficient databases. It is to be noted that these are not mutually exclusive and should be evaluated in combination with each other.

Data usage: While designing a data model, emphasis should be laid on the patterns that the applications will be using to access the data. The patterns refer to reading, writing, updating and deletion of data. Some applications are completely read-centric (like the reporting application), while others are write-centric like an e-commerce application. Some are a combination of both. In some applications, a particular feature is read-heavy while others are write-heavy. There are possibilities that even within a single document some data is frequently updated while other data remains static. Based on these patterns, appropriate strategies should be devised using relationships, indexes, growth in document size and atomicity. Documents with Embedded relationships perform better than documents with References relationships if both the data are needed while reading.

Atomicity: Atomicity in database parlance means that operations either succeed or fail as a single unit. If there are multiple sub-operations within a parent transaction, the parent operation will fail if any of its sub-transactions fail. Operations in MongoDB happen at the collection level. A single write operation can affect only a single collection. Even if it attempts to affect multiple collections, these will be treated as separate operations. There is no support from the database engine to roll back a part of operations, if the sub-operations fail. The application should implement the logic for affecting multiple collections.

If updating multiple collections is a requirement, Embedded relationships should be used because entire data is available within a single document. There is no risk that a part of the operation will succeed or fail. However, References relationships can be used when it does not matter if sub-operations fail.

Document structure: Document structure plays a crucial role in data Modelling. The application is written based on the structure of documents. The documents can be designed using the References or Embedment relationship.

Document growth: MongoDB assigns a fixed document size during the initialisation phase. MongoDB's storage engine will relocate the document on the disk when document size exceeds the allocated space for that document, MongoDB will relocate the document on the disk. With MongoDB 3.0.0, however, the default use of the power of two-sized allocations minimises the occurrences of such re-allocations as well as

allows for the effective reuse of the freed record space.

When using Embedded documents, it should be carefully analysed if the sub-object can grow out of bounds. If it can, there is the possibility of performance degradation when the size of the document crosses its limit. In such cases, References relationship should be used to ensure that growth in document size stays within limits.

Indexing: Indexes are especially useful in improving performance while retrieving data. They help in fetching sorted data, helping applications to eliminate the need to sort them explicitly. Collections that are frequently accessed for read operations should implement indexes on the column on which frequent searches are made. While indexes are beneficial during read operations, they introduce negative performance for write operations. Indexes should be built on columns that are updated infrequently and queried frequently. Another drawback of indexes is that they consume additional storage space and should be considered carefully before being implemented.

Sharding: Sharding is a database load balancing technique fully supported by MongoDB. It refers to horizontal partitioning of data into multiple MongoDB instances, with each instance holding specific and unique data. Each instance is referred to as a 'shard' and hosts a portion of overall collection data. Sharding is typically employed with large datasets in collections with heavy operations on them.

Strategies for MongoDB data Modelling

Data Modelling is equally important in the NoSQL world as it is in the relational world. There are important facets of applications that cannot be realised without implementing a proper and optimised data model.

One-to-one with Embedded relationship: In this strategy, one data entity is embedded into another data entity, where both the entities have a one-to-one relationship with each other.

An example of a one-to-one Embedded relationship, between the user and the details of his place of birth, is illustrated here:

```
{
  "_id": "Ritesh",
  "name": "Ritesh Modi",
  "placeOfBirth": {
    "street": "123 xyz Street",
    "city": "xyzcity",
    "state": "xyzState",
    "zip": "12345"
  }
}
```

A one-to-one Embedded relationship should be used when:

- Both the name and place of birth are retrieved together frequently.